

1. Introduction to Corpus and its Types

- **Corpus (plural: Corpora):**
A large, structured set of texts used for linguistic analysis and NLP model training.
Example: Brown Corpus, Gutenberg Corpus, Wikipedia Dump.
- **Types of Corpora:**
 1. **Monolingual Corpus** – texts in one language.
 2. **Multilingual Corpus** – texts in multiple languages.
 3. **Parallel Corpus** – same text translated into different languages.
 4. **Tagged Corpus** – annotated with POS tags, named entities, etc.
 5. **Specialized Corpus** – domain-specific (medical, legal, etc.).

Key Characteristics:

Machine-Readable: Can be processed by a computer.

Structured: Often annotated with additional information (e.g., POS tags, syntax trees).

Representative: Aims to be a balanced sample of a particular language or genre.

Types of Corpora:

Monolingual: Contains texts from one language (e.g., the Brown Corpus of American English).

Multilingual: Contains texts from more than one language.

Parallel: A type of multilingual corpus where texts in one language are aligned with their translations in another (e.g., Canadian Hansards, EU proceedings). Crucial for machine translation.

Annotated (Tagged): Corpus that has been enriched with additional linguistic information.

Part-of-Speech (POS) Tagged: Each word is tagged with its grammatical category (noun, verb, etc.).

Parsed (Treebank): Sentences are annotated with syntactic structure (e.g., the Penn Treebank).

Diachronic vs. Synchronic: Diachronic corpora track language change over time (e.g., Google Ngrams), while synchronic corpora represent a single time period.

General vs. Specialized: General corpora (e.g., the Corpus of Contemporary American English - COCA) cover many genres, while specialized corpora focus on a specific domain (e.g., medical journals, legal documents).

Components of NLP

NLP is typically divided into two main components that often work together in a pipeline:

Natural Language Understanding (NLU): The "reading" component. It involves mapping input text to useful representations and analyzing aspects like:

- Syntactic Parsing (Grammar)
- Semantic Analysis (Meaning)
- Discourse Integration (Linking sentences)

Goal: To understand the literal meaning and intent behind the text.

Natural Language Generation (NLG): The "writing" component. It involves generating meaningful phrases and sentences from internal representations.

- Text Planning
- Sentence Planning
- Realization

Goal: To produce human-readable text from data.

Common NLP Tasks: Machine Translation, Sentiment Analysis, Speech Recognition, Chatbots, Text Summarization, etc.

Regular Expressions (Regex)

Definition: A powerful sequence of characters that defines a search pattern. Used for string matching, finding, and replacing.

Used for:

- Pattern matching in text
- Searching & replacing text
- Text validation (emails, phone numbers, etc.)

Symbol	Meaning	Example	Matches
.	Any single character	c.t	cat, cut, cot
^	Start of string	^Hello	Hello world (✓), Hi Hello (X)
\$	End of string	end\$	the end (✓), ending (X)
*	0 or more occurrences	go*	g, go, goo, gooo
+	1 or more occurrences	go+	go, goo, gooo (not g)
?	0 or 1 occurrence (optional)	colou?r	color, colour
{n}	Exactly n times	\d{3}	123
{n,}	n or more times	\d{2,}	12, 1234
{n,m}	Between n and m times	\d{2,4}	12, 123, 1234

2. Character Classes

Class	Meaning	Example
[abc]	Matches a, b, or c	"cat" → c
[^abc]	Not a, b, or c	"dog" → d
[a-z]	Lowercase letters	
[A-Z]	Uppercase letters	
[0-9]	Digits	

3. Predefined Character Sets

Shorthand	Meaning
\d	Digit (0–9)
\D	Non-digit
\w	Word character (a-z, A-Z, 0-9, _)
\W	Non-word character
\s	Whitespace (space, tab, newline)
\S	Non-whitespace

Useful Examples

- **Email:** `^[\\w\\.\\-]+@[\\w\\.\\-]+\\.\\w+$`
- **Phone number (10 digits):** `^\\d{10}$`
- **Postal Code (6 digits):** `^\\d{6}$`
- **Only alphabets:** `^[A-Za-z]+$`

Applications in NLP

- Tokenization (splitting text)
- Cleaning data (removing special characters, HTML tags)
- Extracting patterns (dates, emails, hashtags)
- Rule-based text classification

4. Corpora

What is a Corpus?

- **Corpus (plural: Corpora)** = A large, structured collection of texts used for linguistic research and NLP tasks.
- Serves as the **knowledge base** for training, testing, and evaluating NLP models.

Example: `from nltk.corpus import gutenberg`

Types of Corpora

1. **Monolingual Corpus**
 - Texts in a single language.
 - Example: Brown Corpus (English).
2. **Multilingual Corpus**
 - Texts in multiple languages (not necessarily translations).
3. **Parallel Corpus**
 - Same text translated into multiple languages.
 - Example: Europarl Corpus (European Parliament proceedings).
4. **Tagged/Annotated Corpus**
 - Includes linguistic annotations (POS tags, syntax, semantics).
 - Example: Penn Treebank.
5. **Spoken Corpus**
 - Transcriptions of spoken language.
 - Example: Switchboard Corpus (telephone conversations).
6. **Domain-Specific Corpus**
 - Specialized texts for a particular field (medical, legal, financial).
7. **Web Corpus**
 - Collected from internet sources (Wikipedia dumps, Common Crawl).

Applications of Corpora in NLP

- **Training machine learning models** (language models, embeddings).
- **POS tagging & parsing.**
- **Machine translation** (using parallel corpora).
- **Speech recognition** (spoken corpora).
- **Information retrieval & text mining.**
- **Sentiment analysis** (domain corpora like product reviews).

Word Tokenization

Definition: Word tokenization is the foundational step in the NLP pipeline where a continuous sequence of characters (a string of text) is split into smaller units called tokens. In most cases, these tokens are words, numbers, or punctuation marks.

Analogy: Imagine taking a sentence and cutting it up with scissors between each word. Each piece you're left with is a token.

Input (Text): "I don't like green eggs and ham!"

Output (Tokens): ["I", "do", "n't", "like", "green", "eggs", "and", "ham", "!"]

Why is it so Important?

- You can't analyze text without first defining what the basic units are. Almost all subsequent NLP tasks rely on tokens:
- Text Pre processing: Stemming, lemmatization, and stop word removal all operate on tokens.
- Frequency Analysis: Building a Bag-of-Words model requires counting tokens.
- Model Training: Machine Learning models (like those for sentiment analysis) learn from sequences of tokens.
- Part-of-Speech (POS) Tagging & Parsing: These tasks assign tags and structure to tokens.

The Core Challenge: It's Not Just Spaces

While splitting on spaces seems like an easy solution (`text.split(" ")`), it fails miserably in many common cases. A good tokenizer must handle linguistic nuances.

Challenge Category	Example	Simple Space Split Fails	What a Good Tokenizer Should Do
Contractions	don't, I'll, we're	['don't']	Split meaningfully: ['do', 'n't'] or ['do', '', 't']
Hyphenated Words	state-of-the-art, pre-trained	['state-of-the-art']	Often keep as one token: ['state-of-the-art']
Punctuation	Hello, world! Is it 10:30?	['Hello,', 'world!', 'Is', 'it', '10:30?']	Separate punctuation: ['Hello', ',', 'world', '!', ...]
Multilingual Text	¿Cómo estás? 今日は	Fails completely	Use language-specific rules.
Emojis & Hashtags	I love NLP! 🐼 #NLPisCool	['I', 'love', 'NLP!', '🐼', '#NLPisCool']	Keep emojis and hashtags as single tokens.
Numbers & Dates	It costs \$19.99 on 12/05/2023.	['It', 'costs', '\$19.99', 'on', '12/05/2023.']	Handle complex numbers and dates.

Common Methods & Algorithms

Whitespace Tokenization:

- The simplest method: `text.split(" ")`
- Pros: Very fast.
- Cons: Fails on all the challenges listed above. Not suitable for real-world applications.

Rule-Based Tokenization (using Regular Expressions):

- Uses a set of hand-crafted rules (often implemented with Regex) to decide where to split.
- Example Rule: "Split on spaces, but not if the period is part of an abbreviation like Dr. or etc.".
- Pros: Transparent and customizable for specific domains.
- Cons: Creating a robust set of rules for all cases is extremely difficult and time-consuming.

Statistical/Machine Learning Tokenization:

- Trains a model to predict the boundaries between tokens.
- Example: The Byte-Pair Encoding (BPE) algorithm, used in many modern models (like GPT), starts with individual characters and iteratively merges the most frequent pairs to create a vocabulary of subword tokens. This is excellent for handling unknown words (OOV problem).
- Pros: Highly accurate, can handle unknown words well.
- Cons: Requires training data and computational resources.

Dictionary-Based Tokenization:

- Relies on a pre-existing dictionary of words. The algorithm tries to find the longest possible sequence of characters that matches a word in the dictionary.
- Commonly used in languages without clear word boundaries, like Chinese and Japanese.
- Pros: Accurate for languages with well-defined dictionaries.
- Cons: Fails for words not in the dictionary (neologisms, slang, misspellings).

Challenges in Word Tokenization

- **Punctuation:** "Let's go!" → Should "Let's" be 1 or 2 tokens?
- **Compound Words:** "ice-cream" → one word or two?
- **Different Languages:**
 - English uses spaces → easy.
 - Chinese, Japanese, Thai → no spaces → need special tokenizers.
- **Abbreviations & Emojis:** "Dr." / "😊" need correct handling.

Tools & Libraries

- **NLTK (Python)** → `word_tokenize("I love NLP.")`
- **spaCy** → `nlp("I love NLP.").text`
- **HuggingFace Tokenizers** → for modern deep learning models
- **Moses Tokenizer** → used in machine translation

Sentence Segmentation (Sentence Tokenization)

Definition

The process of dividing a continuous text into its constituent sentences. It's the first step in building a pipeline for many tasks like sentiment analysis or machine translation, which often operate on single sentences.

The Challenge: It's Not Just Splitting on Periods

A period (.) is ambiguous. It can mark:

- The end of a sentence.
- An abbreviation (e.g., Dr., etc., Inc., U.S.A.).
- A decimal point (e.g., It costs \$19.99).
- An ellipsis (...).

A naive `text.split(". ")` would incorrectly split "He works at Ph.D. He is smart." into ["He works at Ph", "D. He is smart."].

How is it Done?

Modern sentence segmentation uses a combination of techniques:

1. **Rule-Based Heuristics:** A set of hand-crafted rules.

Capitalization: Check if the character after the period is capitalized. (He is Dr. Smith. He is here.-> likely a sentence break).

- **Dictionary of Abbreviations:** Maintain a list of common abbreviations to avoid splitting on them.
 - **Length of Potential Sentence:** Very short "sentences" are likely false splits.
2. **Machine Learning Classifiers (Most Common):** A binary classification model is trained to predict whether a period (or other punctuation like ?, !) is a sentence boundary.

Features used for training:

- The word before the punctuation (is it a known abbreviation?).
- The word after the punctuation (is it capitalized?).
- The length of the words and sentences.
- The presence of other punctuation.

Tools like spaCy and NLTK's punkt module use pre-trained models for highly accurate segmentation.

Minimum Edit Distance

What is it?

The Minimum Edit Distance (MED) between two strings is the minimum number of operations required to transform one string into the other.

Operations: The three standard operations are:

1. **Insertion (of a character)**
2. **Deletion (of a character)**
3. **Substitution (of one character for another)**

Also known as: Levenshtein Distance, which is a specific case where Insertion, Deletion, and Substitution all have a cost of 1.

Formula (Dynamic Programming):

- $D(i, j) = \min(D(i-1, j) + 1, \text{// deletion}$
- $D(i, j-1) + 1, \text{// insertion}$
- $D(i-1, j-1) + \text{cost}) \text{ // substitution}$

Example:

- Word1 = "kitten", Word2 = "sitting"
- Operations:
 - kitten → sitten (substitution k→s)
 - sitten → sittin (substitution e→i)
 - sittin → sitting (insertion g)
- **Edit Distance = 3**

Why is it Useful?

- **Spell Checking:** Finding the closest correct word to a misspelling (e.g., giraffe -> giraffe, MED=3?).
- **Plagiarism Detection:** Measuring the similarity between two documents.
- **Computational Biology:** Aligning DNA, RNA, or protein sequences (where the "alphabet" is {A, T, G, C}).
- **Machine Translation & Speech Recognition:** Evaluating output quality (e.g., Word Error Rate is based on edit distance).

One-Hot Encoding

- **The Fundamental Problem:** The Vocabulary Mismatch Human Language: Discrete, symbolic, categorical. Words are distinct units (e.g., "cat", "dog", "the").
- **Machine Learning Models:** Require numerical, continuous vectors as input. They perform mathematical operations (addition, multiplication, etc.) on numbers.
- **The Challenge:** How do we convert words, which are categorical variables, into a numerical format that a machine can understand?

One-Hot Encoding is the simplest and most intuitive solution to this problem.

What is One-Hot Encoding?

Definition: A method for converting categorical variables into a binary vector representation where:

- Each category (word) in the vocabulary is represented as a unique vector.
- The vector is all zeros except for a single '1' at the index corresponding to that specific category.
- **Analogy:** Imagine a light switch panel for every word in the dictionary. To represent the word "cat", you turn only the "cat" switch to ON (1) and leave all the others OFF (0).

The Step-by-Step Process

Let's encode the sentence: "The cat sat on the mat."

Step 1: Build the Vocabulary

1. Tokenize the text: ["the", "cat", "sat", "on", "the", "mat"]
 2. Identify all unique words and sort them (alphabetically is common). This forms our vocabulary (V).
- Vocabulary: ['cat', 'mat', 'on', 'sat', 'the']

- Vocabulary Size (V) = 5

Step 2: Assign an Index

Assign a unique integer index to each word in the vocabulary.

Word	Index
Cat	0
Mat	1
on	2
sat	3
the	4

Step 3: Create the Vectors

For each word, create a binary vector of length V (5). The element at the word's index is set to 1; all others are 0.

Word	One-Hot Vector (length=5)
Cat	[1, 0, 0, 0, 0]
Ma t	[0, 1, 0, 0, 0]
On	[0, 0, 1, 0, 0]
sat	[0, 0, 0, 1, 0]
the	[0, 0, 0, 0, 1]

The entire sentence can now be represented as a sequence of these one-hot vectors.

Key Characteristics & Implications

1. High Dimensionality:

- The length of each vector is the size of the entire vocabulary (V).
- For a large corpus (e.g., all English Wikipedia articles), V can easily be 100,000 or even 1,000,000. This creates vectors with 100,000+ dimensions for just one word.

2. Sparsity:

- The vectors are overwhelmingly composed of zeros. For any given word, only one element is non-zero.
- This is a very sparse representation. Storing all these zeros is computationally inefficient.

3. Orthogonality & No Semantic Similarity:

- All word vectors are orthogonal (perpendicular) to each other. The dot product between any two different word vectors is 0.
- This is the biggest drawback. It implies that all words are equally different from each other.

The distance between cat and dog is the same as the distance between cat and pizza.

The model has no way of learning that "cat" and "dog" are both pets and are more similar to each other than to "pizza".

5. Advantages and Disadvantages

Advantages

Simple & Intuitive: Easy to understand and implement.

Preserves Word Identity: No information about the word is lost in translation.

Works with any classifier: Provides a baseline numerical input.

Disadvantages

High Dimensionality: Leads to the "curse of dimensionality," making models slow and hungry for memory.

Sparsity: Inefficient storage and computation.

No Semantic Meaning: Fails to capture any relationships between words (synonymy, relatedness).

Vocabulary-Based: It can only encode words seen during the vocabulary creation phase. New words (Out-of-Vocabulary words) cannot be handled.

1. Bag of Words (BoW)

Concept

An extension of one-hot encoding from the word-level to the document-level. It represents a document by the multiset (bag) of its words, disregarding grammar and word order but keeping track of frequency.

How it Works

1. Step 1: Create a vocabulary V from all unique words in the entire corpus.
2. Step 2: For each document, create a vector of length V .
3. Step 3: Each element in the vector counts how many times the corresponding word from the vocabulary appears in the document.

Example

- * Doc1: "The cat sat on the mat."
- * Doc2: "The dog sat on the log."
- * Vocabulary: $['cat', 'dog', 'sat', 'on', 'the', 'mat', 'log']$ ($V=7$)
- * BoW(Doc1): $[1, 0, 1, 1, 2, 1, 0]$ // "the" appears 2 times
- * BoW(Doc2): $[0, 1, 1, 1, 2, 0, 1]$

Characteristics

- * Pros: Simple to understand and implement. Creates a fixed-length representation for documents of varying sizes.
- * Cons:
 - * Loss of Order: "The cat chased the dog" and "The dog chased the cat" have identical BoW representations.

- * Sparsity: Documents will only use a small subset of the vocabulary, leading to vectors full of zeros.

- * Semantic Blindness: Does not capture meaning or relationships between words. "Love" and "adore" are treated as completely different.

2. N-grams (Bi-gram and Tri-gram)

Concept

A contiguous sequence of n items from a given text. Items can be characters, syllables, or most commonly, words.

- * Unigram (1-gram): Single words. This is the standard BoW model. (the, cat, sat)

- * Bigram (2-gram): Pairs of consecutive words. (the cat, cat sat, sat on)

- * Trigram (3-gram): Triplets of consecutive words. (the cat sat, cat sat on)

Why Use Them?

They capture a limited amount of local word order and context. This can help mitigate the "semantic blindness" of unigrams.

- * The phrases "strong coffee" and "strong opinion" have different meanings. A unigram model only sees "strong". A bigram model sees the distinct pairs "strong coffee" and "strong opinion".

The Trade-off: The Vocabulary Explosion

The vocabulary size grows exponentially with n .

- * Unigram Vocab Size: V

- * Bigram Vocab Size: $\sim V^2$

- * Trigram Vocab Size: $\sim V^3$

This leads to extreme sparsity and computational challenges. There's a trade-off between context and manageability.

3. TF-IDF (Term Frequency-Inverse Document Frequency)

Concept

A statistical measure that evaluates how important a word is to a document in a collection (corpus). It is a weighted version of BoW, designed to downweight common, uninformative words.

It consists of two parts:

1. TF (Term Frequency): How **often** a word appears in a document.

* $TF(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total number of terms in document } d)$

2. IDF (Inverse Document Frequency): How **unique** or **rare** a word is across all documents. It scales up the weight of rare words.

* $IDF(t) = \log(\text{Total number of documents} / (\text{Number of documents containing term } t))$

TF-IDF Calculation

$TF-IDF(t, d) = TF(t, d) * IDF(t)$

Intuition & Example

* The word "the" appears very frequently in a document (high TF) but also appears in **every** document (very high document frequency, so very low IDF). $TF-IDF("the")$ will be low. It's not important.

* The word "quantum" appears several times in a specific physics paper (high TF) and rarely appears in other documents (low document frequency, so high IDF). $TF-IDF("quantum")$ will be very high. It is a key term for that document.

Characteristics

- * Pros: Effectively identifies the most discriminative words in a document. Much better than raw counts for tasks like Information Retrieval (search engines) and Document Classification.
- * Cons: Still suffers from the other limitations of BoW (sparsity, no semantic meaning).

4. Word2Vec

The Paradigm Shift: From Sparse to Dense Vectors

Word2Vec, introduced by Google in 2013, was a revolutionary breakthrough. It moves away from sparse, high-dimensional, one-hot-like vectors to dense, low-dimensional, continuous vector space representations called word embeddings.

- * One-Hot / BoW: Vectors are ~20,000+ dimensions, with 99% zeros.
- * Word2Vec: Vectors are typically 100-300 dimensions, with **all** elements being real numbers (not just 0/1).

Core Idea: Distributional Hypothesis

The meaning of a word can be understood by the company it keeps. Words that appear in similar contexts (surrounded by similar neighboring words) have similar meanings.

Word2Vec uses a shallow neural network to learn these embeddings by predicting words from their contexts.

Two Main Architectures

1. Continuous Bag-of-Words (CBOW):

- * Task: Predict a target word (e.g., "fox") given its context words (e.g., "quick", "brown", "jumps").

- * Analogy: "Fill in the blank."

- * Faster, better for frequent words.

2. Skip-gram:

- * Task: Predict the surrounding context words given a target word.

- * Analogy: "Given a word, list the words that might appear around it."

- * Works well with smaller datasets and can represent rare words effectively.

Magic Properties of Word Embeddings

The vector space captures semantic and syntactic relationships. Relationships between words become geometric relationships between vectors.

- * Semantic Analogy: $\text{vector}(\text{"King"}) - \text{vector}(\text{"Man"}) + \text{vector}(\text{"Woman"}) \approx \text{vector}(\text{"Queen"})$

- * Syntactic Analogy: $\text{vector}(\text{"walking"}) - \text{vector}(\text{"walk"}) + \text{vector}(\text{"swim"}) \approx \text{vector}(\text{"swimming"})$

- * Similarity: Similar words are clustered together in the vector space.

Characteristics

- * Pros:

- * Captures complex semantic relationships.

- * Dense vectors are computationally efficient for ML models.

- * Solves the sparsity and dimensionality problems of earlier methods.

- * Cons:

- * Cannot handle out-of-vocabulary (OOV) words (fixed vocabulary).

* Each word has a single vector, ignoring polysemy (words with multiple meanings, e.g., "bank" as a river bank vs. financial bank).